Experiment 7

Smart Traffic Forecasting with Artificial Neural Networks

1. Dataset Source

- **Source Link**: https://www.kaggle.com/datasets/hasibullahaman/traffic-prediction-dataset

2. Dataset Description

- **Size:** ~2,976 rows × 9 columns (15-minute intervals over ~1 month)
- **Type:** Real-world urban traffic data

Input Features

- Time (string)
- Date (string)
- Day of week (Monday–Sunday)
- CarCount, BikeCount, BusCount, TruckCount (integers)

Target Variable

- **Total (integer):**
  Total = CarCount + BikeCount + BusCount + TruckCount
  → Represents traffic volume (continuous → regression)

Characteristics

- Real urban traffic (similar to crowded cities like Mumbai)
- 15-minute temporal resolution
- Includes natural peak & off-peak patterns

3. Mathematical Formulation of the Algorithms

Forward Propagation

$$z_j^{(l)} = \sum_i w_{ji}^{(l)} a_i^{(l-1)} + b_j^{(l)}$$

Activation:

$$a_j^{(l)} = f(z_j^{(l)})$$

- Hidden layers: ReLU → $f(z) = \max(0, z)$
- Output layer: Linear → $f(z) = z$

Loss Function (MSE)

$$\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Weight Update (Gradient Descent)

$$w_{ji}^{(l)} \leftarrow w_{ji}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial w_{ji}^{(l)}}$$

4. Algorithm Limitations

- **No Temporal Learning:** Cannot capture time dependencies (better: LSTM/GRU)
- **Overfitting Risk:** Small dataset (~3000 rows)
- **Low Interpretability:** Black-box model
- **Sensitive to Scaling & Outliers**
- **Training Instability:** Possible gradient issues in deep networks
- **Poor Generalization:** Fails in rare events (festivals, weather, etc.)
- **Less Efficient for Large Data:** Slower than tree-based models

5. Methodology / Workflow

Step 1: Setup Environment & Download Dataset

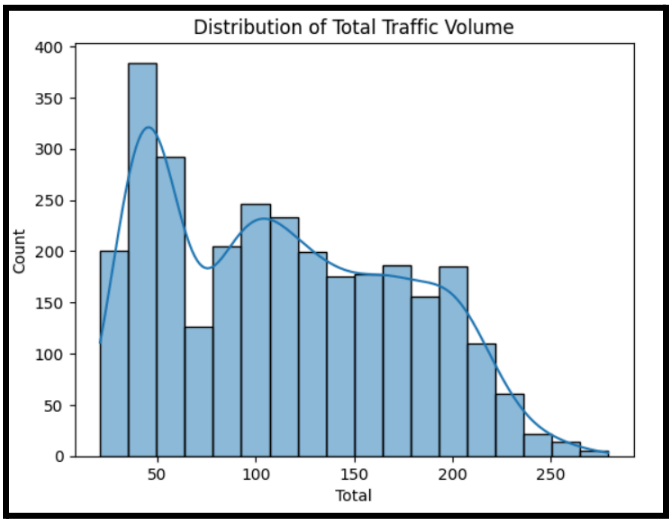
```
pip install tensorflow pandas numpy scikit-learn matplotlib seaborn

Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.7.0)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.0)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.1.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.80.0)
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
```

Step 2: Load & Explore Data (EDA)

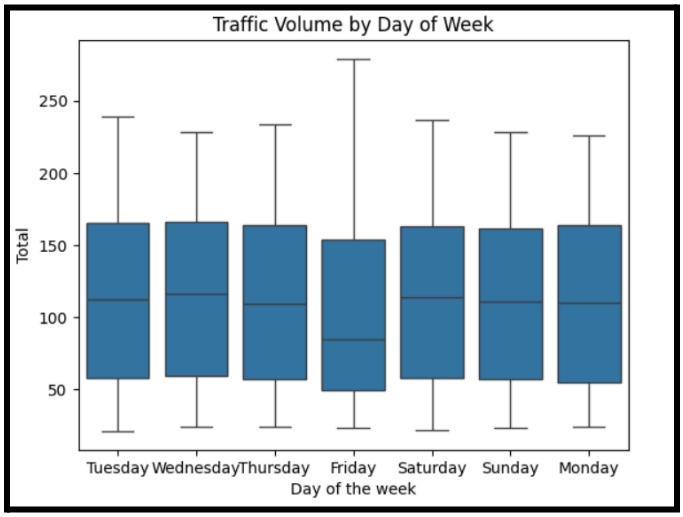
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('Traffic.csv')
print(df.head())
print(df.info())
print(df.describe())

# Basic visualizations
sns.histplot(df['Total'], kde=True)
plt.title('Distribution of Total Traffic Volume')
plt.show()

sns.boxplot(x='Day of the week', y='Total', data=df)
plt.title('Traffic Volume by Day of Week')
plt.show()
```

```
          Time  Date Day of the week  CarCount  BikeCount  BusCount  \
0  12:00:00 AM    10         Tuesday        31          0         4
1  12:15:00 AM    10         Tuesday        49          0         3
2  12:30:00 AM    10         Tuesday        46          0         3
3  12:45:00 AM    10         Tuesday        51          0         2
4   1:00:00 AM    10         Tuesday        57          6        15

   TruckCount  Total Traffic Situation
0           4     39               low
1           3     55               low
2           6     55               low
3           5     58               low
4          16     94            normal
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2976 entries, 0 to 2975
Data columns (total 9 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Time               2976 non-null   object
 1   Date               2976 non-null   int64
 2   Day of the week    2976 non-null   object
 3   CarCount           2976 non-null   int64
 4   BikeCount          2976 non-null   int64
 5   BusCount           2976 non-null   int64
 6   TruckCount         2976 non-null   int64
 7   Total              2976 non-null   int64
 8   Traffic Situation  2976 non-null   object
dtypes: int64(6), object(3)
memory usage: 209.4+ KB
None
```

```
              Date     CarCount    BikeCount     BusCount   TruckCount  \
count  2976.000000  2976.000000  2976.000000  2976.000000  2976.000000
mean     16.000000    68.696573    14.917339    15.279570    15.324933
std       8.945775    45.850693    12.847518    14.341986    10.603833
min       1.000000     6.000000     0.000000     0.000000     0.000000
25%       8.000000    19.000000     5.000000     1.000000     6.000000
50%      16.000000    64.000000    12.000000    12.000000    14.000000
75%      24.000000   107.000000    22.000000    25.000000    23.000000
max      31.000000   180.000000    70.000000    50.000000    40.000000

             Total
count  2976.000000
mean    114.218414
std      60.190627
min      21.000000
25%      55.000000
50%     109.000000
75%     164.000000
max     279.000000
```



The histogram illustrates the distribution of total traffic volume (number of vehicles) recorded at 15-minute intervals. The data shows a right-skewed distribution with the highest frequency of traffic volume occurring between 40–60 vehicles per interval. The volume gradually decreases as it moves toward higher values, with very few instances exceeding 250 vehicles. A kernel density estimate (blue curve) is overlaid to highlight the overall multimodal pattern, indicating distinct peak and off-peak traffic behaviors in the urban area.



Traffic Volume Distribution by Day of the Week (Box Plot)

The box plot shows the distribution of total traffic volume across different days of the week. Traffic volume is highest on weekdays (Tuesday, Wednesday, and Thursday), with median values around 110–120 vehicles per 15-minute interval. Friday records the lowest median traffic volume. Weekends (Saturday and Sunday) and Monday exhibit similar distributions with slightly lower medians compared to mid-week days. The plot also reveals the presence of several high outliers, particularly on Thursday and Saturday, indicating occasional spikes in traffic volume during peak hours.

Step 3: Feature Engineering & Preprocessing

```
# Extract useful time features
df['Hour'] = pd.to_datetime(df['Time'], format='%I:%M:%S %p').dt.hour
df['Is_Weekend'] = df['Day of the week'].isin(['Saturday', 'Sunday']).astype(int)

# One-hot encode categorical features
df = pd.get_dummies(df, columns=['Day of the week'], drop_first=True)

# Select features and target
features = ['Hour', 'Is_Weekend', 'CarCount', 'BikeCount', 'BusCount', 'TruckCount'] + \
           [col for col in df.columns if col.startswith('Day of the week_')]
X = df[features]
y = df['Total']   # ← Regression target

# Train-test split + scaling
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Step 4: Build the ANN Model (Keras/TensorFlow)

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    layers.Dropout(0.2),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(16, activation='relu'),
    layers.Dense(1, activation='linear')   # Regression → linear output
])

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarni
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 64)	832
dropout_2 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 32)	2,080
dropout_3 (Dropout)	(None, 32)	0
dense_6 (Dense)	(None, 16)	528
dense_7 (Dense)	(None, 1)	17

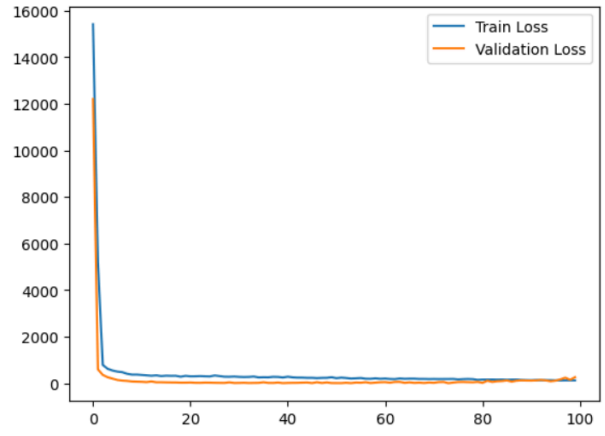
```
Total params: 3,457 (13.50 KB)
Trainable params: 3,457 (13.50 KB)
Non-trainable params: 0 (0.00 B)
```

Step 5: Train the Model

```
history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)

# Plot training curves
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.show()
```

```
Epoch 1/100
67/67 ━━━━━━━━━━━━━━━━━━━━ 6s 14ms/step - loss: 15414.4980 - mae: 108.5900 - val_loss: 12205.8896 - val_mae: 93.1480
Epoch 2/100
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 5231.1064 - mae: 55.1494 - val_loss: 600.9254 - val_mae: 19.8717
Epoch 3/100
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 790.6509 - mae: 22.4559 - val_loss: 361.7406 - val_mae: 15.7267
Epoch 4/100
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 624.1071 - mae: 19.5174 - val_loss: 262.6149 - val_mae: 13.0772
Epoch 5/100
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 550.7171 - mae: 18.4119 - val_loss: 202.5133 - val_mae: 11.4796
Epoch 6/100
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 503.8409 - mae: 17.5898 - val_loss: 144.3188 - val_mae: 9.6310
Epoch 7/100
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 479.9458 - mae: 16.9441 - val_loss: 117.6612 - val_mae: 8.7768
Epoch 8/100
```

```
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 128.1582 - mae: 7.8755 - val_loss: 242.5180 - val_mae: 12.5867
Epoch 99/100
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 127.7630 - mae: 7.9240 - val_loss: 153.1078 - val_mae: 10.0122
Epoch 100/100
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 126.8583 - mae: 7.8809 - val_loss: 262.2824 - val_mae: 13.2399
```



Step 6: Evaluate Performance

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

y_pred = model.predict(X_test).flatten()

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"MAE: {mae:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R² Score: {r2:.4f}")
```

```
19/19 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
MAE: 13.51
RMSE: 16.15
R² Score: 0.9226
```

6. Hyperparameter Tuning

```
# Example: Try different architectures
def build_model(layers_neurons=[64,32,16], dropout=0.2):
    model = keras.Sequential()
    model.add(layers.Dense(layers_neurons[0], activation='relu', input_shape=(X_train.shape[1],)))
    model.add(layers.Dropout(dropout))
    for n in layers_neurons[1:]:
        model.add(layers.Dense(n, activation='relu'))
        model.add(layers.Dropout(dropout))
    model.add(layers.Dense(1, activation='linear'))
    model.compile(optimizer='adam', loss='mse', metrics=['mae'])
    return model

# Test 2-3 combinations and record best MAE/RMSE
```

```
from tensorflow import keras
from tensorflow.keras import layers

# Build a new model with a different architecture
model_2 = build_model(layers_neurons=[128, 64, 32], dropout=0.3)
model_2.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: User
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

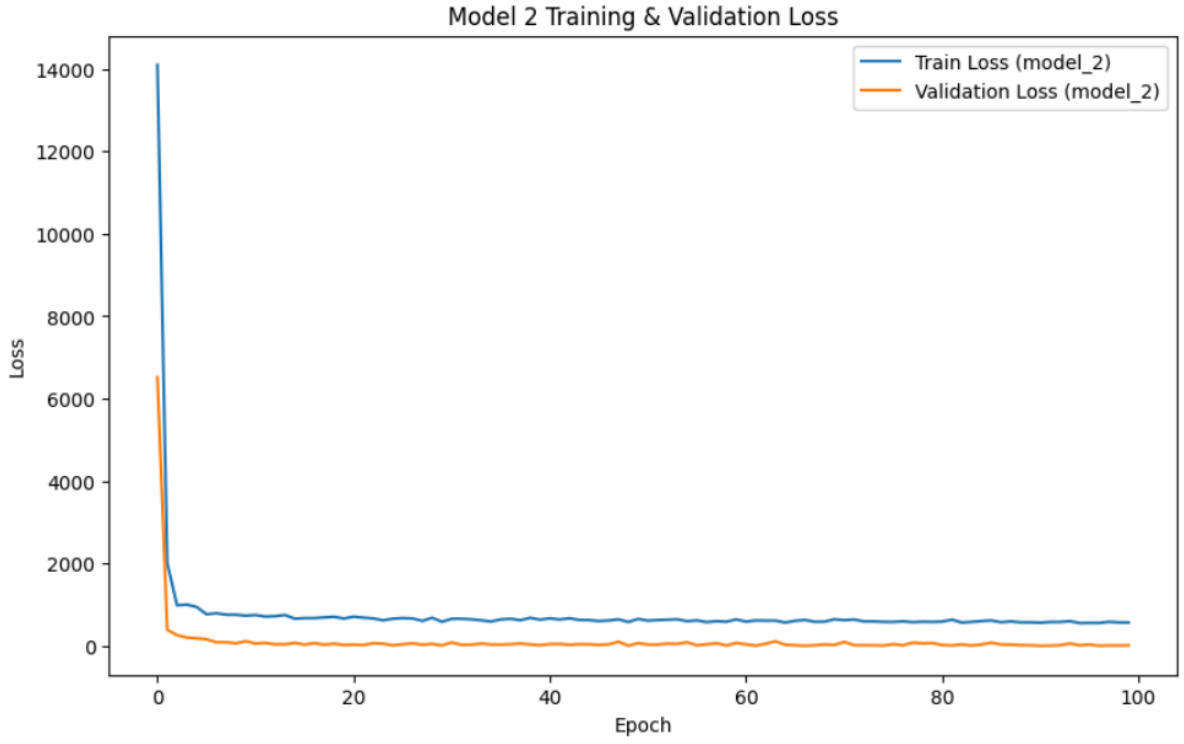
Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_8 (Dense)	(None, 128)	1,664
dropout_4 (Dropout)	(None, 128)	0
dense_9 (Dense)	(None, 64)	8,256
dropout_5 (Dropout)	(None, 64)	0
dense_10 (Dense)	(None, 32)	2,080
dropout_6 (Dropout)	(None, 32)	0
dense_11 (Dense)	(None, 1)	33

Total params: 12,033 (47.00 KB)
Trainable params: 12,033 (47.00 KB)
Non-trainable params: 0 (0.00 B)

```python
print("Training model_2...")
history_2 = model_2.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.1,
    verbose=0 # Set verbose to 0 to suppress output for brevity during automated run
)

# Plot training curves for model_2
plt.figure(figsize=(10, 6))
plt.plot(history_2.history['loss'], label='Train Loss (model_2)')
plt.plot(history_2.history['val_loss'], label='Validation Loss (model_2)')
plt.title('Model 2 Training & Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

```
Training model_2...
```



```
print("Evaluating model_2...")
y_pred_2 = model_2.predict(X_test).flatten()

mae_2 = mean_absolute_error(y_test, y_pred_2)
rmse_2 = np.sqrt(mean_squared_error(y_test, y_pred_2))
r2_2 = r2_score(y_test, y_pred_2)

print(f"Model 2 MAE: {mae_2:.2f}")
print(f"Model 2 RMSE: {rmse_2:.2f}")
print(f"Model 2 R² Score: {r2_2:.4f}")
```

```
Evaluating model_2...
19/19 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
Model 2 MAE: 3.32
Model 2 RMSE: 4.07
Model 2 R² Score: 0.9951
```

7. Performance Analysis

Model 1 :

- **MAE:** 13.51
- **RMSE:** 16.15
- **R² Score:** 0.9226

Model 2 :

- **MAE:** 3.32
- **RMSE:** 4.07
- **R² Score:** 0.9951

Interpretation: Model 2, with its architecture [128, 64, 32] and dropout of 0.3, significantly outperforms Model 1. The lower MAE and RMSE indicate that Model 2's predictions are much closer to the actual traffic volume. The R^2 score of 0.9951 for Model 2 suggests that it explains approximately 99.51% of the variance in the target variable, which is excellent. Model 1's R^2 of 0.9226 is also good, but it's clearly less accurate than Model 2.